



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 226 - Concurrent Systems

Assignment 2 Specification

Concurrent Queues and Producer-Consumer Systems

Release Date: 21 September 2026

Submission Deadline: 14 October 2026

1 Overview

In this assignment, you will implement and investigate two concurrent queue implementations:

- A bounded blocking queue.
- A lock-free queue.

You will use these queues in a producer-consumer system in which producer threads generate jobs and place them into a shared queue, while consumer threads remove and process those jobs.

The assignment consists of three main tasks. You will first implement the two queue algorithms. You will then create producer and consumer threads that use the queues concurrently. Finally, you will compare the behaviour of the two queue implementations using different numbers of producers and consumers.

The emphasis of this assignment is on understanding the differences between lock-based and lock-free concurrent data structures, as well as the effect that producer and consumer workloads can have on the behaviour of a concurrent queue.

2 Learning Outcomes

By completing this assignment you should be able to:

- Implement a bounded concurrent queue using locks and conditions.
- Implement a lock-free queue using atomic operations.
- Explain the use of `compareAndSet()` in lock-free algorithms.
- Explain the purpose of sentinel nodes in concurrent queues.
- Explain how producer and consumer threads interact through a shared queue.

3 Requirements

This assignment is to be completed in groups of **three**. Group members are expected to contribute equally, and all group members should be able to explain any part of the solution during the demonstration. You may structure your solution in any reasonable way, provided that your implementation satisfies the requirements of this assignment.

Both queue implementations must provide the following operations:

```
void enq(T item);  
T deq();
```

The same producer and consumer implementation should be usable with either queue.

4 Task 1 – Queue Implementations

You must implement the following two concurrent queues.

4.1 Bounded Queue

Implement a bounded queue with a fixed maximum capacity.

The queue must:

- Store items in first-in-first-out (FIFO) order.
- Allow multiple producer and consumer threads to access the queue safely.
- Cause an enqueue operation to wait when the queue is full.
- Cause a dequeue operation to wait when the queue is empty.
- Use appropriate locking and condition variables to coordinate producers and consumers.

Your implementation must provide:

```
void enq(T item);  
T deq();
```

You may select a reasonable queue capacity for testing. The same capacity must be used when comparing experimental runs.

4.2 Lock-Free Queue

Implement an unbounded lock-free queue.

The queue must:

- Store items in first-in-first-out (FIFO) order.
- Be safe for use by multiple producer and consumer threads.
- Use atomic references and `compareAndSet()` operations.
- Make use of a sentinel node.
- Correctly maintain the head and tail of the queue during concurrent operations.

- Not use `synchronized`, `ReentrantLock`, or other explicit locking mechanisms to provide synchronization.

Your implementation must provide:

```
void enq(T item);  
T deq();
```

If a dequeue operation is attempted while the lock-free queue is empty, the operation may indicate that no item is currently available. Your consumer implementation must handle this situation correctly.

5 Task 2 – Producer-Consumer System

You will use both queue implementations in a concurrent producer-consumer program.

5.1 Producer

Create a producer thread class.

Each producer must:

- Generate jobs.
- Place generated jobs into the shared queue using `enq()`.
- Continue until it has produced its assigned number of jobs.

Each produced job must have an identifier that allows you to distinguish it from other jobs.

N.B. A `Job` represents a single unit of work passed between producer and consumer threads. Each job must contain a unique identifier.

For the purposes of this assignment, processing a job simply means successfully removing it from the queue and recording that it has been processed. No additional computational task is required.

At the end of an execution, the total number of jobs processed must equal the total number of jobs produced.

5.2 Consumer

Create a consumer thread class.

Each consumer must:

- Remove jobs from the shared queue using `deq()`.
- Process each job that it successfully removes.
- Continue until the required workload has been completed.

Your program must ensure that all produced jobs are eventually consumed.

5.3 Running Program

Create a main program that:

- Creates the selected queue implementation.
- Creates the required number of producer threads.
- Creates the required number of consumer threads.
- Starts all producer and consumer threads.
- Waits for all work to be completed.
- Displays enough information to demonstrate that the program executed correctly.

The program must be able to run using either the bounded queue or the lock-free queue.

6 Task 3 – Experimental Comparison

Conduct a small experiment comparing the bounded queue and lock-free queue under different producer-consumer configurations.

You must test the following configurations:

- 4 producers and 4 consumers.
- 6 producers and 2 consumers.
- 2 producers and 6 consumers.

These configurations represent balanced, producer-heavy, and consumer-heavy workloads respectively. The total number of jobs processed must remain the same for every experiment. The work should therefore be divided between the available producer threads. For example, if

the total workload consists of 12,000 jobs:

- 4 producers would produce 3,000 jobs each.
- 6 producers would produce 2,000 jobs each.
- 2 producers would produce 6,000 jobs each.

You may select a different total workload, provided that the same workload is used for both queue implementations and for all producer-consumer configurations.

For each configuration, record:

- The queue implementation.
- The number of producers.
- The number of consumers.
- The total execution time.
- Throughput i.e. The number of jobs processed per second.

Present your results in a table.

The purpose of this comparison is not simply to determine which queue is faster. You should consider how the behaviour of each queue changes under the different producer-consumer workloads.

7 Demonstration

Students are required to demonstrate their implementation to a tutor during their scheduled practical session.

During the demonstration, students must:

- Demonstrate the bounded queue.
- Demonstrate the lock-free queue.
- Demonstrate producer and consumer threads operating concurrently.
- Demonstrate that all produced jobs are correctly processed.
- Present the results of the experimental comparison.
- Explain relevant parts of their implementation when requested.

8 Upload Checklist

Before submitting, ensure that your group has included all of the following:

- Your bounded queue implementation.
- Your lock-free queue implementation.
- Your producer implementation.
- Your consumer implementation.
- Your main/running program.
- Your experimental results.

You may include additional source files where appropriate.

9 Submission

Your solution will be submitted via **ClickUp**. **All group members are required to submit.** Your submission should include:

- All source code required to compile and run your solution.
- The student numbers of all group members.
- Your experimental results and observations.

All required source files must be placed in a single ZIP archive. The archive must be named according to the following format:

`uXXXXXXXX.zip`

where `XXXXXXXX` is your student number.

No late submissions will be accepted!

10 Marking Rubric

Assessment Item	Marks
Bounded Queue implementation	2
Lock-Free Queue implementation	2
Producer implementation	1
Consumer implementation	1
Running program	1
Experimental comparison and observations	1
Conceptual Question 1	3
Conceptual Question 2	3
Conceptual Question 3	3
Conceptual Question 4	3
Total	20